AGENTIC AI WORKSHOP · DAY 7 OF 10

Multi-agent systems & A2A

Meridian becomes a crew (supervisor, research, writer, critic), and the critic becomes callable by anyone over A2A.

You build: **A crew** · about 3.9 h of material

Contents

1. Split the work the way the business already does — Why this matters · 5 min
2. Supervisor-worker vs peer-to-peer — Theory · 10 min
3. The 2026 framework landscape — Theory · 6 min
4. Lab 1: A supervisor crew in LangGraph — Lab · intermediate · 40 min
5. Lab 2: The same crew in CrewAI — Lab · beginner · 20 min
6. A2A: agents calling agents — Theory · 10 min
7. Lab 3: Make the critic callable over A2A — Lab · intermediate · 35 min
8. Handoffs: agents passing the conversation — Theory · advanced · 8 min
9. Lab 4: A team that hands off — Lab · advanced · 30 min
10. A2A beyond request/response — Theory · advanced · 8 min
11. Lab 5: A critic that streams, asks and returns artifacts — Lab · advanced · 30 min
12. Is the crew worth it? Measure it — Theory · advanced · 5 min
13. Lab 6: One agent vs the crew — Lab · advanced · 20 min
14. Check yourself — Quiz · 5 min
15. Day 7 recap — Recap · 3 min
16. Quiz answer key

WHY THIS MATTERS 5 MIN

Split the work the way the business already does

More agents isn't more intelligence. It's more organisation, so split only where you can name the reason.

Northstar's risk committee wants a **monthly Merchant Risk Report** for ACME, BOLT and CRUX. In the bank the analyst who prepares the report is not the reviewer who signs it off: **segregation of duties**. Today Meridian mirrors that: a **research** agent with data access, a **writer** with none, and an independent **critic**.

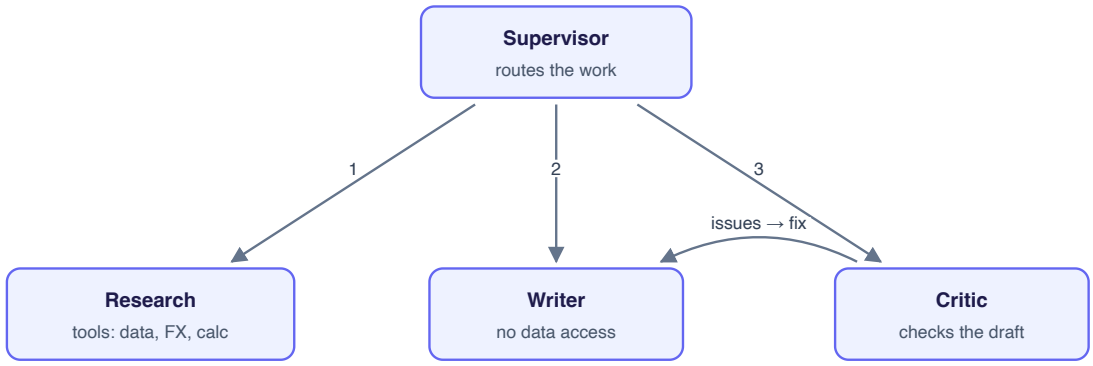
By the end of today you can

- Choose between supervisor-worker and peer-to-peer patterns
- Know when multiple agents earn their extra cost, and when they don't
- Build a supervisor crew in LangGraph, and the same crew in CrewAI
- Publish an agent card and call an agent over A2A from another process

THEORY 10 MIN

Supervisor-worker vs peer-to-peer

A supervisor crew for the monthly report



1. The supervisor sends research first: MTD volumes, FX conversion, verdicts.
2. The writer turns research into a report, and has no tools, so it can't fetch or invent data.
3. An independent critic checks figures, currencies and recommended actions.
4. Issues go back to the writer, with a cap of 2 revisions. Deterministic guards in the supervisor decide when to stop.

	Supervisor-worker	Peer-to-peer (handoffs)
Control	central, explicit	emergent
Audit	one router to inspect	harder: who called whom?
Fit for regulated work	✅	⚠️

Use multiple agents when sub-tasks need different **permissions**, **tools** or **models**, when a clean context per task helps, when work runs in **parallel**, or when different **teams** own the agents. Otherwise one well-tooled agent is cheaper and easier to test. Errors compound: three agents each 95% reliable in a chain ≈ **86%**.

THEORY 6 MIN

The 2026 framework landscape

Frameworks reshuffle; the patterns stay. Learn the patterns and any framework is a weekend.

Framework	Mental model	Good for
LangGraph	explicit state graph	control, persistence, human approval
CrewAI	role-based crew (role, goal, backstory)	fast, readable prototypes
Microsoft Agent Framework	successor to AutoGen + Semantic Kernel	Microsoft / .NET shops
OpenAI Agents SDK	agents + handoffs + guardrails	lightweight handoffs
Google ADK	Agent Development Kit, A2A-native	Google Cloud

LAB INTERMEDIATE 40 MIN

Lab 1: A supervisor crew in LangGraph

Goal. Research, writer and critic agents produce the monthly report under a supervisor.

1. Read the crew

Find these four design decisions:

1. **The writer has** `tools=[]` : it can't fetch or invent data, only rewrite what research found (segregation of duties).
2. **The supervisor's guards run before the LLM**: approved → stop; 2 revisions → stop; no research → research. The LLM only decides the genuinely open question (critic or writer next?).
3. `Command(goto=...)` : each worker reports back to the supervisor; there are no fixed edges between workers.
4. **The revision cap** (`revisions >= 2`): a critic that never approves can't loop forever.

`crew_langgraph.py`

```
"""Day 7 Lab 1: supervisor-worker crew in LangGraph: research + writer + critic -> monthly Merchant
Risk Report.

Run:  python crew_langgraph.py
"""
from typing import Literal

from langchain.agents import create_agent
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command
from pydantic import BaseModel
from typing_extensions import TypedDict

from llm import chat_model
from tools_lc import calculator, get_fx_rate, lookup_merchant

research = create_agent(chat_model(), tools=[calculator, lookup_merchant, get_fx_rate],
                        system_prompt="You are the RESEARCH analyst at Northstar Pay. For each
merchant compute MTD "
                                      "volume in the limit currency (use get_fx_rate and the
calculator), utilisation %, "
                                      "and verdict (OVER_LIMIT >100%, NEAR_LIMIT >90%, else OK).
Output a factual table only.")
writer = create_agent(chat_model(), tools=[],                       # no data access: segregation of
duties
                      system_prompt="You are the REPORT WRITER for Northstar's risk committee. Turn
research findings "
                                    "into a concise report: executive summary, table, limit breaches &
recommended "
                                    "actions. Never invent numbers.")


class Review(BaseModel):
    approved: bool
    issues: list[str]


critic_llm = chat_model(max_tokens=4000).with_structured_output(Review)
CRITIC_RUBRIC = ("Check: (1) every figure matches the research table, (2) currencies are stated and
consistent with "
                 "each limit, (3) every OVER_LIMIT merchant has a recommended action, (4) no invented
```

```
data. "
                "Approve only if all pass.")


class CrewState(TypedDict):
    task: str
    research: str
    draft: str
    review: Review | None
    revisions: int


class Next(BaseModel):
    next: Literal["research", "writer", "critic", "FINISH"]
    reason: str


supervisor_llm = chat_model(max_tokens=2000).with_structured_output(Next)


def supervisor(s: CrewState) -> Command[Literal["research", "writer", "critic", "__end__"]]:
    # Deterministic guards first (business rules in code), then let the LLM decide.
    if s.get("review") and s["review"].approved:
        return Command(goto=END)
    if s.get("revisions", 0) >= 2:
        return Command(goto=END)
    if not s.get("research"):
        return Command(goto="research")
    if not s.get("draft"):
        return Command(goto="writer")
    status = f"research done: True; draft exists: True; last review: {s.get('review')}"
    d = supervisor_llm.invoke(f"Task: {s['task']}\nStatus: {status}\nPick the next worker "
                              "(critic if the draft is unreviewed; writer if the critic found
issues).")
    print(f"  supervisor -> {d.next} ({d.reason[:80]})")
    return Command(goto=END if d.next == "FINISH" else d.next)


def run_research(s: CrewState):
    out = research.invoke({"messages": [("user", s["task"])]}, {"recursion_limit": 40})
    print("  research done")
    return Command(goto="supervisor", update={"research": out["messages"][-1].text})


def run_writer(s: CrewState):
    feedback = f"\n\nCritic feedback to fix:\n{s['review'].issues}" if s.get("review") else ""
    out = writer.invoke({"messages": [("user", f"Research findings:\n{s['research']}{feedback}")]})
    print("  draft written")
    return Command(goto="supervisor", update={"draft": out["messages"][-1].text,
                                              "revisions": s.get("revisions", 0) + (1 if
s.get("review") else 0)})


def run_critic(s: CrewState):
    r = critic_llm.invoke(f"{CRITIC_RUBRIC}\n\nRESEARCH:\n{s['research']}\n\nDRAFT:\n{s['draft']}")
    print(f"  critic: approved={r.approved} issues={r.issues}")
    return Command(goto="supervisor", update={"review": r})


g = StateGraph(CrewState)
g.add_node("supervisor", supervisor)
g.add_node("research", run_research)
g.add_node("writer", run_writer)
g.add_node("critic", run_critic)
g.add_edge(START, "supervisor")
```

```
crew = g.compile()

INITIAL = {"task": "Monthly merchant risk report for ACME, BOLT, CRUX (October 2026).",
           "research": "", "draft": "", "review": None, "revisions": 0}

if __name__ == "__main__":
    out = crew.invoke(INITIAL, {"recursion_limit": 30})
    print("\n" + out["draft"])
    print("\nREVIEW:", out["review"])
```

2. Run it

You'll see `research done`, `draft written`, `critic: approved=...`, and `supervisor -> ...` lines. A run typically makes 5–8 model calls.

RUN `python crew_langgraph.py`

3. Plant an error

Tell the writer to "round all figures to the nearest million". Does the critic catch 9.92M → 10M?

4. Count the cost

How many LLM calls did the crew make, compared with one Meridian agent doing the same job? Was the split worth it? (Lab 6 measures this properly.)

5. If something goes wrong

Symptom	Likely cause
`GraphRecursionError`	the supervisor keeps choosing workers: check the guards, and the `recursion_limit` (30)
Critic never approves	its rubric is impossible (e.g. demands data the research doesn't have): read its `issues`
Wrong numbers in the report	the writer invented them: it has no tools, so it must copy from research. Tighten its prompt

Expected result. A draft report, then a critic review; the run ends when the critic approves or after 2 revisions.

LAB BEGINNER 20 MIN

Lab 2: The same crew in CrewAI

Goal. Feel the difference between a role-based crew and an explicit graph.

1. Use a separate environment

CrewAI pins an older pydantic, so keep it apart from the main course environment.

RUN

```
python3.13 -m venv .venv-crew
.venv-crew/bin/pip install crewai python-dotenv langchain-core
```

2. Read the crew

Roles, goals and backstories read like job descriptions.

`crew_crewai.py`

```
"""Day 7 Lab 2: the same scenario in CrewAI (role-based crew) for architectural contrast.

Separate env (CrewAI pins pydantic<2.13):  python3.13 -m venv .venv-crew && .venv-crew/bin/pip install
crewai python-dotenv langchain-core
Run:  python crew_crewai.py            (uses OPENROUTER_API_KEY; CREWAI_MODEL overrides the model
string)
"""
import json
import os
import sys

from crewai import LLM, Agent, Crew, Process, Task
from crewai.tools import tool
from dotenv import load_dotenv

from data import MERCHANTS, fx_rate

load_dotenv()
llm = LLM(model=os.getenv("CREWAI_MODEL", "openrouter/anthropic/claude-opus-5.5"))   # LiteLLM-style
string


@tool("lookup_merchant")
def lookup_merchant(merchant_id: str) -> str:
    """Merchant file: monthly processing limit (amount+currency), risk tier, MTD settlement batches
(amount + currency)."""
    return json.dumps(MERCHANTS[merchant_id.strip().upper()])


@tool("get_fx_rate")
def get_fx_rate(base: str, quote: str) -> float:
    """Spot FX rate converting 1 unit of base into quote."""
    return fx_rate(base, quote)


@tool("calculator")
def calculator(expression: str) -> float:
    """Evaluate an arithmetic expression exactly."""
    from tools_lc import calculator as calc   # reuse Day 3's safe calculator
    return calc.invoke({"expression": expression})
```

```
researcher = Agent(role="Merchant Risk Research Analyst",
                   goal="Compute MTD volume in limit currency, utilisation and verdict for each
merchant",
                   backstory="Meticulous analyst at Northstar Pay who never skips FX conversion.",
                   tools=[lookup_merchant, get_fx_rate, calculator], llm=llm)
writer = Agent(role="Risk Committee Report Writer",
               goal="Write a concise, accurate committee report from research findings",
               backstory="Former merchant risk officer; writes for busy executives; never invents
numbers.", llm=llm)
critic = Agent(role="Independent Merchant Risk Reviewer",
               goal="Verify figures, currencies and recommended actions; reject reports with any
error",
               backstory="Second line of defence. Applies the four-check rubric strictly.", llm=llm)

t_research = Task(description="Research ACME, BOLT, CRUX for October 2026.", expected_output="A
factual table",
                  agent=researcher)
t_write = Task(description="Draft the committee report.", expected_output="Markdown report",
agent=writer,
               context=[t_research])
t_review = Task(description="Review the draft against the research. List issues or say APPROVED.",
                expected_output="APPROVED or a list of issues", agent=critic, context=[t_research,
t_write])

if __name__ == "__main__":
    process = Process.hierarchical if "--hierarchical" in sys.argv else Process.sequential
    kwargs = {"manager_llm": llm} if process == Process.hierarchical else {}
    crew = Crew(agents=[researcher, writer, critic], tasks=[t_research, t_write, t_review],
                process=process, verbose=True, **kwargs)
    print(crew.kickoff())
```

3. Run it, sequential then hierarchical

RUN

```
.venv-crew/bin/python crew_crewai.py
.venv-crew/bin/python crew_crewai.py --hierarchical
```

4. Fill in the contrast

Lines of code · how routing is decided · capped critic loop? · persistence? · human approval? · which would you ship in a bank?

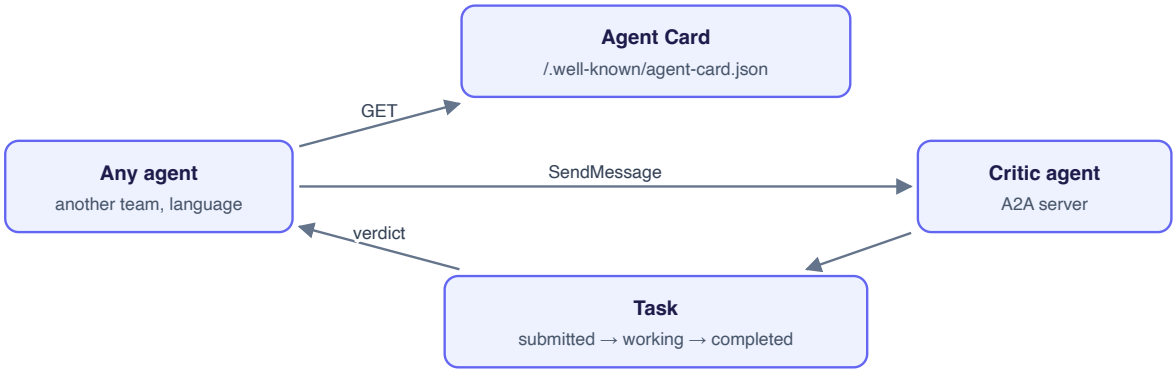
Expected result. Typically: CrewAI is faster to a readable prototype; LangGraph gives the control and audit trail production needs.

THEORY 10 MIN

A2A: agents calling agents

MCP connects an agent to tools and data. A2A lets an agent delegate a task to another agent it doesn't own.

Discover, then delegate



1. Discovery: the client reads the agent card: name, skills, URL and auth, at a standard path.
2. It sends a message containing the draft report. That creates a TASK on the server.
3. The task moves through states: submitted → working → (input-required) → completed or failed.
4. The verdict comes back, without the client knowing anything about the critic's internals.

Tool or agent? A deterministic capability (an FX rate service) belongs behind **MCP**. An autonomous agent with its own reasoning, owner and long-running work belongs behind **A2A**. A2A is now governed by the Agentic AI Foundation, alongside MCP.

LAB INTERMEDIATE 35 MIN

Lab 3: Make the critic callable over A2A

Goal. Publish the critic's agent card, serve it over A2A, and call it from a second, independent process.

1. Wrap the critic as an AgentExecutor

`critic_a2a/agent_executor.py`

```
"""Day 7 Lab 3: the critic wrapped as an A2A AgentExecutor (a2a-sdk 1.x)."""
from a2a.helpers import get_message_text, new_task_from_user_message, new_text_message
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.tasks import TaskUpdater
from a2a.types import TaskState
from pydantic import BaseModel

from llm import chat_model


class Review(BaseModel):
    approved: bool
    issues: list[str]


RUBRIC = ("You are Northstar Pay's independent merchant risk report reviewer. Facts: ACME limit 9.5M
USD with batches "
          "4.2M EUR, 3.1M USD, 2.0M EUR (EUR/USD 1.10); CRUX is High risk tier and on the chargeback
watchlist, so it "
          "is NOT eligible for a limit increase. Check: (1) figures internally consistent and FX-
converted, "
          "(2) currencies stated and consistent with each limit, (3) any limit increase names the
correct approver and "
          "respects eligibility, (4) no unsupported claims.")


class CriticExecutor(AgentExecutor):
    def __init__(self):
        self.llm = chat_model(max_tokens=4000).with_structured_output(Review)

    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
        task = context.current_task or new_task_from_user_message(context.message)
        if not context.current_task:
            await event_queue.enqueue_event(task)
        updater = TaskUpdater(event_queue=event_queue, task_id=task.id, context_id=task.context_id)
        await updater.update_status(state=TaskState.TASK_STATE_WORKING,
                                    message=new_text_message("Reviewing merchant risk report..."))
        report = get_message_text(context.message) or ""
        try:
            review = await self.llm.ainvoke(f"{RUBRIC}\n\nREPORT:\n{report}")
        except Exception as e:  # report failures as a failed task, not a crashed server
            await updater.update_status(state=TaskState.TASK_STATE_FAILED,
message=new_text_message(f"error: {e}"))
            return
        await updater.update_status(state=TaskState.TASK_STATE_COMPLETED,
                                    message=new_text_message(review.model_dump_json()))

    async def cancel(self, context: RequestContext, event_queue: EventQueue) -> None:
        raise NotImplementedError("cancel not supported")
```

2. Publish the agent card and serve it

`critic_a2a/__main__.py`

```
"""Day 7 Lab 3: publish the critic's Agent Card and serve it over A2A (JSON-RPC).

Run (from meridian/):  python -m critic_a2a      -> card at http://127.0.0.1:9999/.well-known/agent-card.json
"""
import os

import uvicorn
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes, create_jsonrpc_routes
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import AgentCapabilities, AgentCard, AgentInterface, AgentSkill
from starlette.applications import Starlette

from critic_a2a.agent_executor import CriticExecutor

HOST, PORT = os.getenv("A2A_HOST", "127.0.0.1"), int(os.getenv("A2A_PORT", "9999"))
PUBLIC_URL = os.getenv("A2A_PUBLIC_URL", f"http://{HOST}:{PORT}")

skill = AgentSkill(
    id="review_merchant_risk_report",
    name="Review merchant risk report",
    description="Independently reviews a merchant risk report for figure, currency, eligibility and approver errors. "
                "Returns JSON {approved: bool, issues: [str]}.",
    input_modes=["text/plain"], output_modes=["application/json", "text/plain"],
    tags=["payments", "merchant-risk", "review"],
    examples=["Review this committee report: ..."],
)

card = AgentCard(
    name="Northstar Risk Report Critic",
    description="Second-line-of-defence reviewer for Northstar Pay merchant risk reports (fictional payment processor).",
    version="1.0.0",
    default_input_modes=["text/plain"], default_output_modes=["application/json"],
    capabilities=AgentCapabilities(streaming=True),
    supported_interfaces=[AgentInterface(protocol_binding="JSONRPC", url=PUBLIC_URL, protocol_version="1.0")],
    skills=[skill],
)

handler = DefaultRequestHandler(agent_executor=CriticExecutor(), task_store=InMemoryTaskStore(), agent_card=card)
app = Starlette(routes=[*create_agent_card_routes(card), *create_jsonrpc_routes(handler, "/")])

if __name__ == "__main__":
    uvicorn.run(app, host=HOST, port=PORT)
```

3. Start the server

`terminal`

```
pip install "a2a-sdk[all]>=1.2,<2" uvicorn httpx
mkdir -p critic_a2a && touch critic_a2a/__init__.py   # save agent_executor.py and __main__.py inside
python -m critic_a2a                 # card: http://127.0.0.1:9999/.well-known/agent-card.json
```

4. Call it from another process

Note the longer HTTP timeout: LLM-backed agents take longer than the default 5 seconds.

client_call.py

```
"""Day 7 Lab 3: discover the critic via its Agent Card and send a task from a second, independent
process.

Prereq: python -m critic_a2a      Run: python a2a_client_call.py
"""
import asyncio
import os

import httpx
from a2a.client import A2ACardResolver, ClientConfig, create_client
from a2a.helpers import get_message_text, get_stream_response_text, new_text_message
from a2a.types import TaskState
from a2a.types import Role, SendMessageRequest

CRITIC_URL = os.getenv("A2A_CRITIC_URL", "http://127.0.0.1:9999")

DRAFT = """ACME MTD volume 9.3M vs limit 9.5M USD (98%) - NEAR_LIMIT.
CRUX MTD volume 820M INR vs limit 800M INR - OVER_LIMIT, increase requested. Approver: Risk Ops
lead."""  # 2 planted errors


async def review(draft: str) -> str:
    async with httpx.AsyncClient() as http:
        card = await A2ACardResolver(httpx_client=http, base_url=CRITIC_URL).get_agent_card()
    print("Discovered:", card.name, [s.id for s in card.skills])
    # LLM-backed agents take longer than httpx's 5 s default: give the client a generous timeout.
    client = await create_client(agent=card, client_config=ClientConfig(
        streaming=False, httpx_client=httpx.AsyncClient(timeout=httpx.Timeout(180.0))))
    req = SendMessageRequest(message=new_text_message(f"Review this committee report:\n{draft}",
role=Role.ROLE_USER))
    texts = []
    async for chunk in client.send_message(req):
        if chunk.HasField("task"):   # non-streaming: the final Task; our verdict is in its status
message
            print("task state:", TaskState.Name(chunk.task.status.state))
            texts.append(get_message_text(chunk.task.status.message))
        else:                        # streaming: message / status / artifact updates
            texts.append(get_stream_response_text(chunk))
    return "\n".join(t for t in texts if t)


if __name__ == "__main__":
    print(asyncio.run(review(DRAFT)))
```

5. Use the remote critic in the crew

Replace `run_critic` in Lab 1 with a node that calls the A2A endpoint. The supervisor neither knows nor cares that it's remote.

6. Read the Agent Card

This is all another team needs to call your agent: name, skills, input/output types, capabilities and the endpoint. Treat it like an API contract: version it, and don't break it.

RUN `curl -s http://127.0.0.1:9999/.well-known/agent-card.json | python -m json.tool`

Expected result. `task state: TASK_STATE_COMPLETED` and a JSON verdict with `approved: false`: ACME wasn't FX-converted, and CRUX isn't eligible for an increase.

THEORY ADVANCED 8 MIN

Handoffs: agents passing the conversation

A supervisor routes every step. With handoffs, the agent that has the conversation decides who should have it next.

	Supervisor	Handoffs ("swarm")
Who routes	one router agent (or code)	each agent, with transfer tools
Good for	fixed workflows, audit, regulated processes	conversations that move between specialists
Risks	a bottleneck; the router's prompt grows	ping-pong between agents; harder to predict
Guard	revision caps, guards in code	a **handoff cap**, and transfer tools only where they make sense

In LangGraph a handoff is a tool that returns `Command(goto="limits_officer", graph=Command.PARENT, update={...})`: "leave this agent's graph and continue at that node in the parent". Pass the messages along, or the next agent starts blind.

Key points

- Give each agent only the transfers it should make: the intake agent can transfer; the officer can't send work back.
- Always cap handoffs and tell the agent what to do when the cap is reached.

LAB ADVANCED 30 MIN

Lab 4: A team that hands off

Goal. Intake, risk analyst and limits officer agents hand the conversation to each other with transfer tools, under a handoff cap.

1. Read the team

- `handoff_to(...)` builds a **tool** per destination. When called it returns a `Command` for the parent graph, carrying the messages, who's active, and the handoff count.
- Each agent gets only the transfers it should make. The limits officer has none: it can't pass work back.
- `IDS` tells agents that "ACME" *is* the ID. Without it, an agent once asked the user for ACME's merchant ID.

handoffs.py

```
# handoffs.py — no supervisor: each agent decides when to hand the conversation to a colleague
("swarm" style).
# Run:  python handoffs.py
from typing import Annotated
from langchain.agents import AgentState, create_agent
from langchain.tools import ToolRuntime, tool
from langchain_core.messages import ToolMessage
from langgraph.graph import START, StateGraph
from langgraph.types import Command
from llm import chat_model
from tools_lc import calculator, get_fx_rate, lookup_merchant

MAX_HANDOFFS = 4                                    # a hard cap: agents must not pass the work around
forever
IDS = "Merchant IDs are short codes: ACME, BOLT, CRUX. Look merchants up with lookup_merchant; never
ask the user for an ID you were given. "

class TeamState(AgentState):
    active: str                                     # who has the conversation now
    handoffs: int

def handoff_to(agent: str, when: str):
    """A tool that moves the conversation to another agent. It returns a Command for the PARENT
graph."""
    @tool(f"transfer_to_{agent}", description=f"Hand the conversation to the {agent} agent. Use when:
{when}")
    def transfer(reason: str, runtime: ToolRuntime) -> Command | str:
        if runtime.state.get("handoffs", 0) >= MAX_HANDOFFS:
            return "Handoff limit reached. Answer the user yourself with what you know, and say what's
missing."
        note = ToolMessage(f"Transferred to {agent}: {reason}", tool_call_id=runtime.tool_call_id)
        print(f"  ↪ {agent}  ({reason[:70]})")
        return Command(goto=agent, graph=Command.PARENT,
                       update={"messages": [*runtime.state["messages"], note], "active": agent,
                               "handoffs": runtime.state.get("handoffs", 0) + 1})
    return transfer

intake = create_agent(chat_model(), state_schema=TeamState, name="intake", tools=[
    handoff_to("risk_analyst", "the question needs merchant data, volumes, FX or limits"),
    handoff_to("limits_officer", "someone asks for a limit increase")],
    system_prompt="You are Northstar Pay's intake assistant. Understand the request, then hand off to
the right "
                  "colleague. Never answer data or policy questions yourself.")

risk_analyst = create_agent(chat_model(), state_schema=TeamState, name="risk_analyst", tools=[
```

```
    lookup_merchant, get_fx_rate, calculator,
    handoff_to("limits_officer", "the merchant is over or near its limit AND an increase was
requested")],
    system_prompt=IDS + "You are a risk analyst. Compute month-to-date volume in the limit currency
with the tools, "
                  "utilisation % and verdict (OVER_LIMIT >100%, NEAR_LIMIT >90%, else OK). Hand off to
the limits "
                  "officer only if an increase was requested; otherwise answer.")

limits_officer = create_agent(chat_model(), state_schema=TeamState, name="limits_officer", tools=
[lookup_merchant],
    system_prompt=IDS + "You are the limits officer. Look up the merchant first. Policy: increases up
to 5% Risk Ops lead; 5–15% merchant risk officer "
                  "(Low/Medium tier) or Head of Merchant Risk (High tier); above 15% Risk Committee;
max 30 days; "
                  "merchants on the chargeback watchlist are NOT eligible. Say whether it's eligible
and who must approve. "
                  "You can't approve anything yourself.")

team = StateGraph(TeamState)
for name, agent in [("intake", intake), ("risk_analyst", risk_analyst), ("limits_officer",
limits_officer)]:
    team.add_node(name, agent, destinations=("intake", "risk_analyst", "limits_officer"))
team.add_edge(START, "intake")
graph = team.compile()

if __name__ == "__main__":
    for request in ["Is BOLT close to its limit this month?",
                    "ACME needs its limit raised to 10.5M USD for Black Friday. Can we do it, and who
signs off?",
                    "CRUX wants +12% for the IPL final."]:
        print(f"\nQ: {request}")
        out = graph.invoke({"messages": [("user", request)], "active": "intake", "handoffs": 0},
{"recursion_limit": 60})
        print(f"[answered by {out['active']} after {out['handoffs']} handoff(s)]\n{out['messages']
[-1].text[:500]}")
```

2. Run the three requests

Request	Expected route	Expected answer
Is BOLT close to its limit?	intake → risk_analyst	about 63%, **OK**
Raise ACME to 10.5M; who signs off?	intake → limits_officer	eligible; ~+10.5%, Medium tier → **merchant risk officer**; max 30 days
CRUX wants +12%	intake → limits_officer	**not eligible**: on the chargeback watchlist

RUN `python handoffs.py`

3. Trigger the cap

Set `MAX_HANDOFFS = 0` and run again. The intake agent can't transfer, gets 'Handoff limit reached', and must answer itself (badly, by design). A cap turns an infinite loop into a degraded but finished answer.

4. Compare with the supervisor

Which pattern would you choose for the monthly report (Lab 1)? For an account manager's chat assistant? Write one sentence for each.

Expected result. Each request lands with the right specialist after one handoff; the ACME answer names the merchant risk officer and the CRUX answer refuses on eligibility.

THEORY ADVANCED 8 MIN

A2A beyond request/response

Feature	What it gives the caller	Server side (a2a-sdk 1.x)
Streaming	progress as it happens: `WORKING: checking figures…`	`updater.start_work(msg)`, `update_status(...)`; card `capabilities.streaming=True`
Artifacts	typed results (JSON, files), separate from chat messages	`updater.add_artifact([new_data_part(...)], name="review")`
Input required	the task pauses and asks; the caller answers **on the same task**	`updater.requires_input(msg)`; client resends with `task_id` / `context_id`
Failure	a failed task with a reason, not a 500	`updater.failed(msg)`
Push notifications	long tasks call your webhook when done	`push_notification_config` on the client
Auth	the card advertises security schemes; calls carry tokens	same as any API: verify, scope, audit (Day 9)

Task states: `SUBMITTED → WORKING → (INPUT_REQUIRED ↔ WORKING) → COMPLETED | FAILED | CANCELED | REJECTED`.

LAB ADVANCED 30 MIN

Lab 5: A critic that streams, asks and returns artifacts

Goal. Upgrade the A2A critic to stream progress, ask for the report when it's missing, and return a typed review artifact.

1. Read the server

Three upgrades, numbered in the code: input-required when there's no report, streamed status, and the review as a `review` artifact.

critic_v2.py

```
# critic_v2.py — the A2A critic, upgraded: streams progress, asks for input when it needs it,
# and returns its review as a structured ARTIFACT. Run:  python critic_v2.py   (http://127.0.0.1:9998)
import re
import uvicorn
from a2a.helpers import get_message_text, new_data_part, new_task_from_user_message, new_text_message
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes, create_jsonrpc_routes
from a2a.server.tasks import InMemoryTaskStore, TaskUpdater
from a2a.types import AgentCapabilities, AgentCard, AgentInterface, AgentSkill
from pydantic import BaseModel
from starlette.applications import Starlette
from llm import chat_model

URL = "http://127.0.0.1:9998"

class Review(BaseModel):
    approved: bool
    issues: list[str]

RUBRIC = ("You are Northstar Pay's independent merchant risk report reviewer. Facts: ACME limit 9.5M
USD, MTD 9.92M USD "
          "after converting 6.2M EUR at 1.10 (OVER_LIMIT, 104.4%); CRUX is High tier and on the
chargeback watchlist, so it "
          "is NOT eligible for a limit increase; 5–15% increases for High tier need the Head of
Merchant Risk. "
          "Check figures, currencies, eligibility and approvers. Approve only if everything is
right.")

class CriticExecutor(AgentExecutor):
    def __init__(self):
        self.llm = chat_model(max_tokens=4000).with_structured_output(Review)

    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
        task = context.current_task or new_task_from_user_message(context.message)
        if not context.current_task:
            await event_queue.enqueue_event(task)                    # a NEW task (first message)
        updater = TaskUpdater(event_queue=event_queue, task_id=task.id, context_id=task.context_id)
        text = get_message_text(context.message) or ""

        # 1. Multi-turn: if there's nothing to review, pause the SAME task and ask for what's missing.
        if not re.search(r"\d", text):
            await updater.requires_input(new_text_message(
                "Send me the report to review: the figures, currencies and any limit-increase
requests.",
                task_id=task.id, context_id=task.context_id))
            return
```

```
        # 2. Streaming progress: the caller sees each status as it happens (if it streams).
        await updater.start_work(new_text_message("Checking figures and currencies…", task_id=task.id,
context_id=task.context_id))
        try:
            review = await self.llm.ainvoke(f"{RUBRIC}\n\nREPORT:\n{text}")
        except Exception as e:                                  # a failed TASK, not a crashed
server
            await updater.failed(new_text_message(f"review failed: {e}", task_id=task.id,
context_id=task.context_id))
            return

        # 3. The result is an ARTIFACT with typed data, not text the caller has to parse.
        await updater.add_artifact([new_data_part(review.model_dump())], name="review")
        verdict = "approved" if review.approved else f"{len(review.issues)} issue(s)"
        await updater.complete(new_text_message(f"Review done: {verdict}", task_id=task.id,
context_id=task.context_id))

    async def cancel(self, context: RequestContext, event_queue: EventQueue) -> None:
        raise NotImplementedError("cancel not supported")

card = AgentCard(
    name="Northstar Risk Report Critic v2",
    description="Reviews merchant risk reports (fictional payment processor). Streams progress;
returns a 'review' artifact "
                "{approved, issues}; asks for the report if you send none.",
    version="2.0.0",
    default_input_modes=["text/plain"], default_output_modes=["application/json"],
    capabilities=AgentCapabilities(streaming=True),
    supported_interfaces=[AgentInterface(protocol_binding="JSONRPC", url=URL,
protocol_version="1.0")],
    skills=[AgentSkill(id="review_merchant_risk_report", name="Review merchant risk report",
                       description="Checks figures, currencies, eligibility and approvers.", tags=
["payments", "risk"],
                       input_modes=["text/plain"], output_modes=["application/json"])],
)

handler = DefaultRequestHandler(agent_executor=CriticExecutor(), task_store=InMemoryTaskStore(),
agent_card=card)
app = Starlette(routes=[*create_agent_card_routes(card), *create_jsonrpc_routes(handler, "/")])

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=9998)
```

2. Start it

RUN `python critic_v2.py`

3. Run the streaming client

Expected:

```
1) Ask without a report:
  task 8f48a644… TASK_STATE_SUBMITTED
  status: TASK_STATE_INPUT_REQUIRED  Send me the report to review: …
2) Answer on the SAME task:
  status: TASK_STATE_WORKING         Checking figures and currencies…
  artifact 'review': {'approved': False, 'issues': [ACME not FX-converted…, CRUX not eligible…, wrong
approver…]}
  status: TASK_STATE_COMPLETED       Review done: 3 issue(s)
```

client_v2.py

```
# client_v2.py — a streaming A2A client that handles input-required and reads the artifact.
# Start critic_v2.py first.   Run:  python client_v2.py
import asyncio
import httpx
from a2a.client import A2ACardResolver, ClientConfig, create_client
from a2a.helpers import get_data_parts, get_message_text, new_text_message
from a2a.types import Role, SendMessageRequest, TaskState

URL = "http://127.0.0.1:9998"
REPORT = """ACME MTD 9.3M vs limit 9.5M USD (98%) - NEAR_LIMIT.
CRUX MTD 820M INR vs 800M INR - OVER_LIMIT; +12% increase recommended, approver Risk Ops lead."""   #
3 planted errors

async def send(client, text, task_id=None, context_id=None):
    """Send one message and print every streamed event. Returns (task_id, context_id, final state)."""
    msg = new_text_message(text, role=Role.ROLE_USER, task_id=task_id, context_id=context_id)
    state = None
    async for event in client.send_message(SendMessageRequest(message=msg)):
        if event.HasField("task"):
            task_id, context_id, state = event.task.id, event.task.context_id, event.task.status.state
            print(f"  task {task_id[:8]}… {TaskState.Name(state)}")
        elif event.HasField("status_update"):
            u = event.status_update
            state = u.status.state
            print(f"  status: {TaskState.Name(state):<26} {get_message_text(u.status.message)}")
        elif event.HasField("artifact_update"):
            a = event.artifact_update.artifact
            print(f"  artifact '{a.name}':", get_data_parts(a.parts)[0])
    return task_id, context_id, state

async def main():
    async with httpx.AsyncClient() as http:
        card = await A2ACardResolver(httpx_client=http, base_url=URL).get_agent_card()
    print(f"Discovered: {card.name} (streaming={card.capabilities.streaming})")
    client = await create_client(agent=card, client_config=ClientConfig(
        streaming=True, httpx_client=httpx.AsyncClient(timeout=httpx.Timeout(300.0))))

    print("\n1) Ask without a report:")
    task_id, context_id, state = await send(client, "Can you review our committee report?")
    if state == TaskState.TASK_STATE_INPUT_REQUIRED:
        print("\n2) Answer on the SAME task:")
        await send(client, f"Here it is:\n{REPORT}", task_id=task_id, context_id=context_id)

asyncio.run(main())
```

RUN `python client_v2.py`

4. Answer on a NEW task instead

Change step 2 to send without `task_id` / `context_id`. The critic starts a fresh task and never connects your answer to the question it asked. Multi-turn A2A depends on reusing the IDs.

5. Fix the report

Correct the three planted errors in `REPORT` and run again. The artifact should say `approved: True`. If it doesn't, read its issues: is the critic right?

Expected result. The task pauses as INPUT_REQUIRED, resumes on the same task, streams WORKING, and returns a review artifact listing all 3 planted errors.

THEORY ADVANCED 5 MIN

Is the crew worth it? Measure it

More agents means more calls, more tokens, more latency and more places to fail. Before keeping a crew, run the same task both ways and compare **cost**, **time** and **quality** (graded against a known answer).

Measured on the DGX test model for the monthly report:

Setup	Tokens	Time	Verdicts right	Actions for breaches
one agent	8,325	106 s	3/3	yes
supervisor crew	10,159	140 s	3/3	yes

Same quality, **+22% tokens, +33% time**. On this task the crew is justified only by **governance**: an independent critic and a writer with no data access. That can be a perfectly good reason, as long as you say so.

LAB ADVANCED 20 MIN

Lab 6: One agent vs the crew

Goal. Run the same report with one agent and with the Lab 1 crew; compare tokens, time and a graded quality score.

1. Read and run

`get_usage_metadata_callback()` adds up tokens across every model call made inside the `with` block, including the crew's internal agents. The grader checks verdicts against a known answer key.

measure.py

```
# measure.py — is the crew worth it? Same task, one agent vs the Lab 1 crew: tokens, time, and an
independent grade.
# Run:  python measure.py      (makes many model calls: a few minutes, and real tokens)
import time
from langchain.agents import create_agent
from langchain_core.callbacks import get_usage_metadata_callback
from pydantic import BaseModel
from crew_langgraph import INITIAL, crew                    # Lab 1
from llm import chat_model
from tools_lc import calculator, get_fx_rate, lookup_merchant

TASK = INITIAL["task"]
single = create_agent(chat_model(), tools=[calculator, lookup_merchant, get_fx_rate], system_prompt=(
    "You are Northstar Pay's merchant risk analyst. Compute each merchant's MTD volume in its limit
currency with the tools, "
    "utilisation % and verdict (OVER_LIMIT >100%, NEAR_LIMIT >90%, else OK), then write a short
committee report: "
    "summary, table, and recommended actions for any breach. Never invent numbers."))

class Grade(BaseModel):
    correct_verdicts: int                                   # out of 3
    has_actions_for_breaches: bool

grader = chat_model(max_tokens=2000).with_structured_output(Grade)
KEY = "Correct: ACME 9.92M USD 104.4% OVER_LIMIT; BOLT 3.15M GBP 63.1% OK; CRUX 820M INR 102.5%
OVER_LIMIT (on chargeback watchlist)."

def run(name, fn):
    start = time.perf_counter()
    with get_usage_metadata_callback() as usage:            # counts tokens across every model call
inside
        report = fn()
    seconds = time.perf_counter() - start
    tokens = sum(u["total_tokens"] for u in usage.usage_metadata.values())
    g = grader.invoke(f"{KEY}\n\nREPORT:\n{report}\n\nHow many of the 3 verdicts are correct? Are
actions given for breaches?")
    print(f"{name:<8} {tokens:>8,} tokens  {seconds:>6.1f}s   verdicts {g.correct_verdicts}/3
actions: {g.has_actions_for_breaches}")

if __name__ == "__main__":
    run("single", lambda: single.invoke({"messages": [("user", TASK)]}, {"recursion_limit": 40})
["messages"][-1].text)
    run("crew", lambda: crew.invoke(INITIAL, {"recursion_limit": 30})["draft"])
```

RUN `python measure.py`

2. **Read your numbers**

   Fill in a table like the one above with your model. Then decide: would you ship the crew for this report? What would have to be true for it to win on quality?

3. **Make the task harder**

   Add 'and recommend whether each breaching merchant should get a temporary increase, with the approver' to the task. Run again. Does the crew's critic now catch errors the single agent makes?

Expected result. A two-line comparison of tokens, seconds and graded verdicts; on the base task both get 3/3 and the crew costs more.

QUIZ 5 MIN

Check yourself

1. **Which is a good reason to split into multiple agents?**
   A. It sounds advanced
   B. Different permissions for different jobs (segregation of duties)
   C. More agents are always smarter
   D. To use more tokens

2. **Three agents, each 95% reliable, in sequence. Overall reliability?**
   A. 95%
   B. About 86%
   C. 100%
   D. About 50%

3. **Where does an A2A client find out what an agent can do?**
   A. In its source code
   B. Its Agent Card at /.well-known/agent-card.json
   C. In the prompt
   D. On a wiki

4. **Northstar's FX-rate service: MCP tool or A2A agent?**
   A. A2A agent
   B. MCP tool

5. **Why does the writer agent have no tools?**
   A. To save cost
   B. Segregation of duties: it can't fetch or invent data
   C. Writers can't use tools
   D. It's a bug

6. **In LangGraph, how does a handoff tool move the conversation to another agent?**
   A. It prints a message
   B. It returns Command(goto=agent, graph=Command.PARENT, update=...)
   C. It calls the agent directly
   D. It restarts the graph

7. **An A2A task is INPUT_REQUIRED. How do you answer it?**
   A. Send a new message with no IDs
   B. Send a message with the same task_id and context_id
   C. Restart the server
   D. Poll until it completes

8. **The crew gets the same score as one agent but costs 22% more. When is it still worth it?**
   A. Never
   B. When governance needs it: an independent reviewer, segregation of duties
   C. Always
   D. When it's faster

9. **What stops two agents passing a conversation back and forth forever?**
    A. Nothing
    B. A handoff cap in code, plus giving agents only the transfers they need
    C. A bigger model
    D. Streaming

RECAP 3 MIN

Day 7 recap

- Start with one agent; split only when you can name the reason.
- Supervisor-worker suits regulated workflows: clear accountability, one router to audit.
- Frameworks change; patterns (supervisor, handoff, critic, approval) don't.
- MCP = agent ↔ tools/data. A2A = agent ↔ agent.
- Handoffs need caps; A2A tasks stream, ask for input and return artifacts; measure a crew against one agent before keeping it.

Next: Day 8: trace, evaluate and cost-track the crew, so 'it demoed well' becomes 'it's measured'.

Quiz answer key

1. **B. Different permissions for different jobs (segregation of duties)**
   Name the reason: permissions, context, parallelism or ownership.
2. **B. About 86%**
   $0.95^3 \approx 0.86$: errors compound.
3. **B. Its Agent Card at /.well-known/agent-card.json**
   Discovery is standardised.
4. **B. MCP tool**
   It's a deterministic capability, not an autonomous agent.
5. **B. Segregation of duties: it can't fetch or invent data**
   It mirrors the controls the business already has.
6. **B. It returns Command(goto=agent, graph=Command.PARENT, update=...)**
   The Command leaves the current agent's graph and continues at the target node in the parent.
7. **B. Send a message with the same task_id and context_id**
   Reusing the IDs continues the same task; without them you start a new one.
8. **B. When governance needs it: an independent reviewer, segregation of duties**
   A legitimate reason, but name it; don't assume more agents means better answers.
9. **B. A handoff cap in code, plus giving agents only the transfers they need**
   Caps turn infinite loops into finished (if imperfect) answers.